

Aplicações *práticas*

Iteradores e geradores brilham quando os dados são grandes, chegam aos poucos ou passam por várias etapas. Nesta aula, seis situações do dia a dia: ler um arquivo linha a linha, contar erros num log com um pipeline, somar um CSV sem carregá-lo inteiro, enviar dados em lotes, comparar valores vizinhos e buscar uma API página por página só enquanto for preciso.

Arquivo linha a linha

Pipeline de log

CSV preguiçoso

batched

pairwise

Paginação

Formação Claude Code 2026 · Baseado em docs.python.org (Reading and Writing Files; csv.DictReader; collections.Counter e defaultdict; itertools.batched e pairwise; Assignment expressions), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em [exemplos/](#) · Saídas reais

Ler um arquivo *linha a linha*

ler_arquivo.py

```
def linhas(caminho):
    with open(caminho, encoding="utf-8") as arq:
        for linha in arq:          # o arquivo já é um iterador
            yield linha.rstrip("\n")

if __name__ == "__main__":
    for n, linha in enumerate(linhas("servidor.log"), 1):
        if n > 3:
            break
        print(n, linha)
```

SAÍDA

```
1 2026-10-05 09:00:01 INFO  login ok usuario=ana
2 2026-10-05 09:00:07 ERROR 500 falha no banco
3 2026-10-05 09:01:12 INFO  login ok usuario=bia
```

- ✓ Um arquivo aberto já é um iterador: o `for` lê uma linha por vez
- ✓ Com um gerador, a leitura fica reaproveitável e o `with` fecha o arquivo no fim
- ✓ Um log de 10 GB ocupa na memória só a linha atual
- ✓ O `break` para a leitura sem ler o resto do arquivo

O `if __name__ == "__main__"` deixa importar a função `linhas` em outro exemplo sem rodar esta demonstração.

Contando erros *num pipeline*

erros_log.py

```
from collections import Counter
from ler_arquivo import linhas

def so_erros(itens):
    return (l for l in itens if " ERROR " in l)

def codigos(itens):
    for linha in itens:
        yield linha.split()[3]      # data hora NIVEL CODIGO

contagem = Counter(codigos(so_erros(linhas("servidor.log"))))
print(contagem)
print(contagem.most_common(1))
```

SAÍDA

```
Counter({'500': 2, '404': 2})
[('500', 2)]
```

servidor.log

```
2026-10-05 09:00:01 INFO  login ok usuario=ana
2026-10-05 09:00:07 ERROR 500 falha no banco
2026-10-05 09:01:12 INFO  login ok usuario=bia
2026-10-05 09:02:30 ERROR 404 pagina /promo
2026-10-05 09:03:44 WARN  disco em 85%
2026-10-05 09:04:02 ERROR 500 falha no banco
2026-10-05 09:05:19 ERROR 404 pagina /antiga
2026-10-05 09:06:55 INFO  logout usuario=ana
```

- ✓ Ler → filtrar **ERROR** → extrair o código: três etapas encadeadas
- ✓ **Counter** consome o gerador e conta cada código

Somando um CSV *sem carregá-lo inteiro*

csv_preguicoso.py

```
import csv
from collections import defaultdict

def vendas(caminho):
    with open(caminho, encoding="utf-8", newline="") as arq:
        for registro in csv.DictReader(arq):
            registro["valor"] = float(registro["valor"])
            yield registro

total = defaultdict(float)
for v in vendas("vendas.csv"):
    total[v["categoria"]] += v["valor"]
print(dict(total))
```

vendas.csv

```
data,categoria,valor
2026-10-01,livros,59.90
2026-10-01,jogos,199.00
2026-10-02,livros,34.50
2026-10-02,cursos,297.00
2026-10-03,jogos,89.90
```

- ✓ `csv.DictReader` também é um iterador: um dicionário por linha
- ✓ O gerador converte o valor para número antes de entregar
- ✓ `defaultdict(float)` começa cada categoria em `0.0`

SAÍDA

```
{'livros': 94.4, 'jogos': 288.9, 'cursos': 297.0}
```

Enviando *em lotes*

lotes.py

```
from itertools import batched

pedidos = [f"P{n}" for n in range(1, 8)]

for lote in batched(pedidos, 3):      # Python 3.12+
    print("enviando", lote)
```

SAÍDA

```
enviando ('P1', 'P2', 'P3')
enviando ('P4', 'P5', 'P6')
enviando ('P7',)
```

- ✓ `batched(iterável, n)` entrega tuplas de até `n` itens
- ✓ O último lote pode ser menor
- ✓ Uso típico: gravar no banco ou chamar uma API de 100 em 100 registros

`batched` existe a partir do Python 3.12. Em versões antigas, use `islice` num laço.

Comparando *com o anterior*

variacao.py

```
from itertools import pairwise

cotacoes = [5.10, 5.18, 5.05, 5.05, 5.22]

for antes, depois in pairwise(cotacoes):
    dif = depois - antes
    seta = "↑" if dif > 0 else "↓" if dif < 0 else "="
    print(f"{antes:.2f} -> {depois:.2f} {seta}")
```

SAÍDA

```
5.10 -> 5.18 ↑
5.18 -> 5.05 ↓
5.05 -> 5.05 =
5.05 -> 5.22 ↑
```

- ✓ `pairwise` entrega os pares vizinhos: (1º, 2º), (2º, 3º)...
- ✓ Sem índices nem `range(len(...))`
- ✓ Serve para variações de preço, intervalos entre datas, diferenças entre leituras de sensor

As cotações são valores de exemplo.

Buscar páginas *só quando precisa*

`paginas.py`

```
BANCO = [f"cliente{n}" for n in range(1, 8)]

def buscar_pagina(numero, tamanho=3):
    """Simula uma API que devolve uma página por vez."""
    print(f" (pedindo página {numero})")
    inicio = (numero - 1) * tamanho
    return BANCO[inicio:inicio + tamanho]

def todos_os_clientes():
    numero = 1
    while pagina := buscar_pagina(numero):
        yield from pagina
        numero += 1

for cliente in todos_os_clientes():
    print(cliente)
    if cliente == "cliente4":
        break                                     # nem pede a página 3
```

SAÍDA

```
(pedindo página 1)
cliente1
cliente2
cliente3
(pedindo página 2)
cliente4
```

- ✓ Quem usa `todos_os_clientes()` não sabe que existem páginas
- ✓ `while pagina := ...` busca e testa numa linha: para quando vier uma página vazia
- ✓ Como o `for` parou no `cliente4`, a página 3 nunca foi pedida

Em APIs reais, isso economiza requisições, tempo e cota de uso.

Média das *notas válidas*

Enunciado: um arquivo tem uma nota por linha, mas também linhas vazias e textos. Escreva um gerador que entregue só as notas válidas e calcule a média.

Uma resposta possível

```
import io

arquivo = io.StringIO("8.5\n\n7\nausente\n10\n6.5\n")

def notas_validas(linhas):
    for linha in linhas:
        try:
            yield float(linha)
        except ValueError:
            continue           # ignora vazias e texto

notas = list(notas_validas(arquivo))
print(notas)
print(f"média: {sum(notas) / len(notas):.2f}")
```

SAÍDA

```
[8.5, 7.0, 10.0, 6.5]
média: 8.00
```

- ✓ `io.StringIO` simula o arquivo; com `open()` funciona igual
- ✓ `float()` lança `ValueError` para vazio e texto: o gerador pula a linha
- ✓ Desafio: aceite também nota com vírgula, como `7,5`

Teste o que *ficou*

1. Por que ler um arquivo grande com for linha in arq?

A) É mais bonito

B) Só a linha atual fica na memória

C) Lê de trás para frente

D) Ordena as linhas

B.

2. list(batched("abcde", 2)) vale:

A) ["ab", "cd", "e"]

B) [("a","b"), ("c","d"), ("e",)]

C) [("a","b"), ("c","d")]

D) Erro

B.

3. pairwise([1, 2, 3]) entrega:

A) (1, 2) e (2, 3)

B) (1, 2) e (3,)

C) (1, 3)

D) [1, 2, 3]

A.

4. No exemplo da paginação, por que a página 3 não foi pedida?

A) Ela estava vazia

B) O for parou antes, e o gerador não avançou

C) A API falhou

D) O limite era 2

B.

Dados grandes, *memória pequena*

Arquivos, `csv.DictReader` e respostas paginadas são fontes naturais de iteradores. Encadeando geradores, você lê, filtra e transforma um item por vez; `Counter` e `defaultdict` agregam o resultado; `batched` agrupa em lotes e `pairwise` compara vizinhos. E nada é calculado além do que o consumidor pede.

for linha in arq

Pipeline

DictReader

batched · pairwise

Paginação

Próxima aula → 06 · Usando Regex

Fontes: docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files; docs.python.org/3/library/csv.html#csv.DictReader; docs.python.org/3/library/collections.html (`Counter`, `defaultdict`); docs.python.org/3/library/itertools.html (`batched`, `pairwise`); docs.python.org/3/reference/expressions.html#assignment-expressions; docs.python.org/3/library/io.html#io.StringIO, consultados em outubro de 2026. Exemplos em `exemplos/`, executados com Python 3.14.4.